



**UNIVERSIDAD AUTÓNOMA DE
TAMAULIPAS**

FACULTAD DE INGENIERÍA Y CIENCIAS

DEPARTAMENTO DE POSGRADO

Minería de texto

Maestría en Ciencias e Ingeniería de Datos

Proyecto final:
**Creación de Mapa Conceptual a partir de un
texto de entrada (Hiperónimos/Hipónimos)**

FECHA DE ENTREGA: 10 DE ABRIL DEL 2025

Profesor:

Dr. Ana Bertha Rios Alvarado

Alumno:

José Eliuth Torres Ortiz

Índice

1. Introducción	3
2. Funciones del Sistema	3
3. Importaciones de Librerías	3
4. Descarga de Recursos	4
5. Código del Sistema	4
5.1. Preprocesamiento del Texto	4
5.2. Extracción de Relaciones de Hiperonimia e Hiponimia	5
5.3. Generación de Mapas Conceptuales	6
5.4. Generación del Código Mermaid	6
5.5. Interfaz Visual con Streamlit	7
6. Implementación del sistema	8
7. Resultados	10
8. Prueba del Sistema	10
9. Conclusiones	12

1. Introducción

El objetivo de este proyecto es crear un sistema de análisis de texto que identifique relaciones de **hiperónimos** e **hipónimos** utilizando expresiones regulares, análisis semántico y dependencias gramaticales. Este sistema genera mapas conceptuales a partir del texto procesado.

2. Funciones del Sistema

El sistema realiza diversas funciones para extraer y visualizar relaciones jerárquicas entre conceptos en el texto:

- **Preprocesamiento del texto:** Limpieza de texto como quitar signos, puntuación, palabras vacías, y conversión a minúsculas.
- **Extracción de relaciones de hiperónimos e hipónimos:**
 - **Expresiones regulares:** Extracción de patrones como "X es un tipo de Y".
 - **Análisis semántico con WordNet.**
 - **Análisis de dependencia:** Con **SpaCy** se extraen las relaciones basadas en dependencias gramaticales.
- **Generación de mapas conceptuales:** Usando las librerías **networkx** y **matplotlib**.
- **Generación de código Mermaid:** Permite usar herramientas web online para ver el mapa conceptual creado.
- **Interfaz de usuario interactiva:** Con la librería **Streamlit** para crear interfaz de usuario interactiva y visualizar los resultados.

3. Importaciones de Librerías

Las siguientes librerías son esenciales para el funcionamiento del sistema:

- **nltk:** Para trabajar con WordNet y obtener relaciones semánticas (hiperonimia/hiponimia).
- **networkx y matplotlib:** Para generar y visualizar los mapas conceptuales.
- **re (expresiones regulares):** Para buscar patrones en el texto.
- **streamlit:** Para crear la interfaz de usuario interactiva.

- **spaCy:** Para la tokenización y extracción de relaciones de dependencia.
- **Pillow:** Para el manejo de imágenes.

4. Descarga de Recursos

Para trabajar con WordNet y obtener los hiperónimos e hipónimos:

- **nltk.download('wordnet'):** Para trabajar con WordNet y obtener las relaciones de hiperonimia e hiponimia.
- **Modelo de SpaCy:** Para realizar la tokenización y análisis de dependencias (en_core_web_sm).

5. Código del Sistema

El sistema se estructura en diversas funciones que trabajan de manera conjunta:

5.1. Preprocesamiento del Texto

Limpieza de los hipónimos extraídos del texto, eliminando palabras de parada y manteniendo solo sustantivos y nombres propios.

```
def clean_hyponyms(hyponyms_text):  
    doc = nlp(hyponyms_text)  
    clean = []  
    for token in doc:  
        if token.pos_ in ["NOUN", "PROPN"]:  
            if token.text.lower() not in custom_stopwords:  
                clean.append(token.lemma_.lower())  
    return list(set(clean))
```

Figura 1: Esta imagen muestra la implementación de la función `clean_hyponyms`.

5.2. Extracción de Relaciones de Hiperonimia e Hiponimia

Se utilizan expresiones regulares para identificar patrones como:

- X is a hypernym of Y
- X such as Y
- Y categorized as X

Estas expresiones se usan para extraer las relaciones entre los conceptos.

En esta imagen se muestran los patrones de expresiones regulares utilizados para identificar relaciones de hiperonimia e hiponimia en el texto, tales como "X es un tipo de Y." "X incluye a Y".

```
patterns = [
    r"(?P<hypernym>\w+)\s+is\s+a\s+hypernym\s+of\s+(?P<hyponyms>[\w\s,]+)",
    r"(?P<hypernym>\w+)\s+such\s+as\s+(?P<hyponyms>[\w\s,]+)",
    r"(?P<hyponyms>[\w\s,]+)\s+is\s+a\s+type\s+of\s+(?P<hypernym>\w+)",
    r"(?P<hypernym>\w+)\s+including\s+(?P<hyponyms>[\w\s,]+)",
    r"(?P<hyponyms>[\w\s,]+)\s+is\s+a\s+kind\s+of\s+(?P<hypernym>\w+)",
    r"(?P<hypernym>\w+)\s+consists\s+of\s+(?P<hyponyms>[\w\s,]+)",
    r"(?P<hypernym>\w+)\s+and\s+other\s+(?P<hyponyms>\w+)",
    r"(?P<hypernym>\w+)\s+or\s+similar\s+(?P<hyponyms>\w+)",
    r"(?P<hyponyms>\w+)\s+belong\s+to\s+the\s+category\s+of\s+(?P<hypernym>\w+)",
    r"(?P<hyponyms>\w+)\s+categorized\s+as\s+(?P<hypernym>\w+)",
    r"examples\s+of\s+(?P<hypernym>\w+)\s+include\s+(?P<hyponyms>[\w\s,]+)",
    r"(?P<hypernym>\w+)\s+divided\s+into\s+(?P<hyponyms>[\w\s,]+)",
    r"(?P<hypernym>\w+)\s+characterized\s+by\s+(?P<hyponyms>[\w\s,]+)",
    r"(?P<hypernym>\w+)\s+comprising\s+(?P<hyponyms>[\w\s,]+)",
    r"(?P<hypernym>\w+)\s+like\s+(?P<hyponyms>[\w\s,]+)"
]
```

Figura 2: Patrones de Expresiones Regulares.

Esta imagen presenta la función que extrae los pares de hiperonimia e hiponimia a partir de los patrones de expresiones regulares y los limpia mediante la función `clean_hyponyms`.

```
def extract_hyponym_patterns(text):
    pairs = []
    for pattern in patterns:
        matches = re.finditer(pattern, text, re.IGNORECASE)
        for match in matches:
            hyper = match.group("hypernym").strip().lower()
            hypos_raw = match.group("hyponyms")
            hypos_clean = clean_hyponyms(hypos_raw)
            for hypo in hypos_clean:
                pairs.append((hyper, hypo))
    return pairs
```

Figura 3: Función `extract_hyponym_patterns`.

5.3. Generación de Mapas Conceptuales

Se utiliza la librería `networkx` para crear nodos representando los conceptos y las relaciones entre ellos. Estos mapas se visualizan con `matplotlib`.

La imagen muestra la función `build_and_save_concept_maps`, que crea y guarda mapas conceptuales para cada relación extraída. Utiliza la librería `networkx` para representar las relaciones como un grafo dirigido.

```
def build_and_save_concept_maps(grouped_pairs):
    root_node = "Conceptual Map"
    global_graph = nx.DiGraph()

    saved_images = []

    # Crear cada relación individualmente y agregar hip
    for hypernym, hyponyms in grouped_pairs.items():
        G = nx.DiGraph()
        for hypo in hyponyms:
            G.add_edge(hypernym, hypo)
            #
            global_graph.add_edge(root_node, hypernym)
            global_graph.add_edge(hypernym, hypo)
```

Figura 4: Generación de Mapas Conceptuales.

5.4. Generación del Código Mermaid

Se genera un diagrama conceptual en formato `Mermaid` que permite representar las relaciones entre los conceptos de manera interactiva.

Aquí se muestra la función `generate_mermaid_code`, que convierte los pares de hiperonimia e hiponimia extraídos en un diagrama representado en formato `Mermaid`. Este código se puede utilizar para generar mapas conceptuales visuales.

```
def generate_mermaid_code(grouped_pairs):
    mermaid_code = "graph TD\n"
    mermaid_code += "    Conceptual_Map\n"

    # Agregar relaciones entre Conceptual Map y cada hiperónimo
    for hypernym, hyponyms in grouped_pairs.items():
        mermaid_code += f"    Conceptual_Map --> {hypernym}\n"
        for hypo in hyponyms:
            mermaid_code += f"    {hypernym} --> {hypo}\n" #

    return mermaid_code
```

Figura 5: Generación de Código Mermaid.

5.5. Interfaz Visual con Streamlit

Esta captura de pantalla muestra la interfaz de usuario en **Streamlit**, que permite al usuario ingresar un texto y generar mapas conceptuales basados en las relaciones de hiperonimia e hiponimia.

```
def main():
    st.title("🧠 Concept Map Generator (Text Mining using Regular Expressions)")

    input_text = st.text_area("📄 Paste or type your text here:", height=200)

    if st.button("🚀 Generate Concept Maps"):
        if not input_text.strip():
            st.warning("Please enter a text to analyze.")
            return
```

Figura 6: Interfaz de Usuario en Streamlit.

En esta imagen se muestra cómo los mapas conceptuales generados son visualizados en la interfaz de usuario, con la opción de descargar o ver los resultados.

```
# Mostrar texto después de preprocesamiento
cleaned_text = ' '.join(clean_hyponyms(input_text))
st.subheader("📄 Processed Text (after cleaning):")
st.write(cleaned_text) # Muestra el texto limpio debajo del cuadro de texto

pairs = extract_hyponym_patterns(input_text)

if not pairs:
    st.info("No hypernym/hyponym relationships found.")
else:
    grouped = defaultdict(list)
    for hyper, hypo in pairs:
        grouped[hyper].append(hypo)

    st.subheader(f"🧠 Relationships found: {len(grouped)}")
    for hyper, hypos in grouped.items():
        st.markdown(f"*{hyper.capitalize()}* → {' '.join(hypos)}")

    st.subheader("🗺️ Concept Maps Generated:")
    saved_files = build_and_save_concept_maps(grouped)
    for filename in saved_files:
        if os.path.exists(filename):
            st.image(Image.open(filename), caption=filename, use_container_width=True)
```

Figura 7: Visualización de los Mapas Conceptuales.

Esta imagen muestra cómo se genera el código **Mermaid** para representar los mapas conceptuales, el cual puede ser visualizado en herramientas externas como el **Mermaid Live Editor**.

```
# Generate Mermaid code
mermaid_code = generate_mermaid_code(grouped)
st.subheader("📄 Mermaid Code of the Concept Map:")
st.code(mermaid_code, language="mermaid")

# Add link to Mermaid viewer
mermaid_viewer_url = "https://mermaid-js.github.io/mermaid-live-editor"
st.markdown(f"[View the concept map in the Mermaid viewer"])
```

Figura 8: Código Mermaid Generado.

6. Implementación del sistema

En esta sección se presentan los resultados obtenidos a partir de la implementación del sistema para la generación de mapas conceptuales utilizando relaciones de **hiperonimia** e **hiponimia**.

El proyecto fue gestionado y subido a **GitHub**, lo que permitió un control de versiones eficiente y la posibilidad de compartir el código con otros colaboradores. El repositorio de **GitHub** albergó todo el código fuente, así como los archivos de configuración necesarios para la ejecución del sistema en diferentes entornos.



 inter097	Actualizar código para cargar modelo SpaCy...	659f99f · yesterday	🕒 9 Commits
	spacy_model/en_core_web_s...	Incluir modelo SpaCy local en el ...	yesterday
	app.py	Actualizar código para cargar m...	yesterday
	requirements.txt	Actualizar código para cargar m...	yesterday

Figura 9: Código subido a github.

El sistema utiliza el modelo de **SpaCy** denominado **en_core_web_sm**, el cual fue cargado en el modelo y utilizado para lograr el correcto funcionamiento.

Gracias a **Streamlit**, se logró crear una interfaz de usuario interactiva que permitió subir el proyecto a un entorno de **producción** o **implementación**. **Streamlit** facilitó la creación de una aplicación web donde los usuarios pueden ingresar un texto, visualizar los mapas conceptuales generados, y explorar las relaciones de **hiperonimia** e **hiponimia** extraídas del texto en tiempo real. Este enfoque de desarrollo permitió llevar el proyecto desde un entorno de desarrollo local a una aplicación web accesible desde cualquier navegador.



Figura 10: Entorno de desarrollo implementado en línea.

En resumen, la implementación del sistema fue exitosa tanto en términos de la ejecución del código como en su despliegue en un entorno de producción utilizando **GitHub** y **Streamlit**, lo que facilitó la interacción del usuario final con los mapas conceptuales generados a partir del texto procesado.

7. Resultados

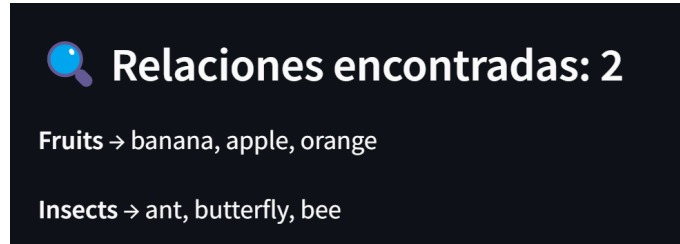


Figura 11: Hiperonimos e hiponimos obtneidos.

8. Prueba del Sistema

En esta sección se presentan los resultados obtenidos a partir de la implementación del sistema. A continuación se muestran los mapas conceptuales generados, que representan las relaciones de **hiperonimia** e **hiponimia** entre diferentes conceptos extraídos del texto:

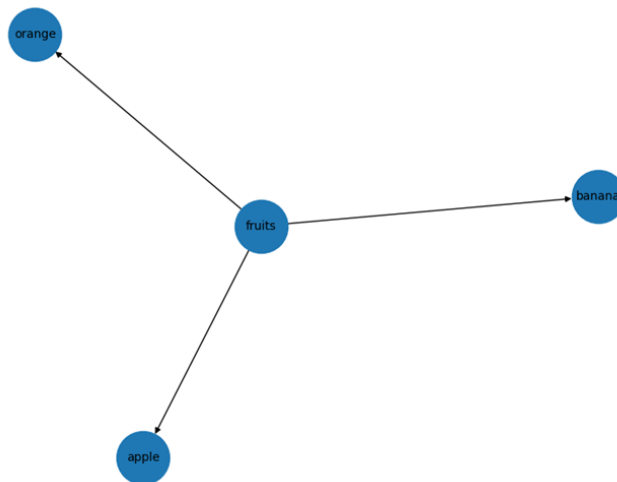


Figura 12: Mapa Conceptual de Frutas: Relación entre frutas como "orange", "banana", y "apple".

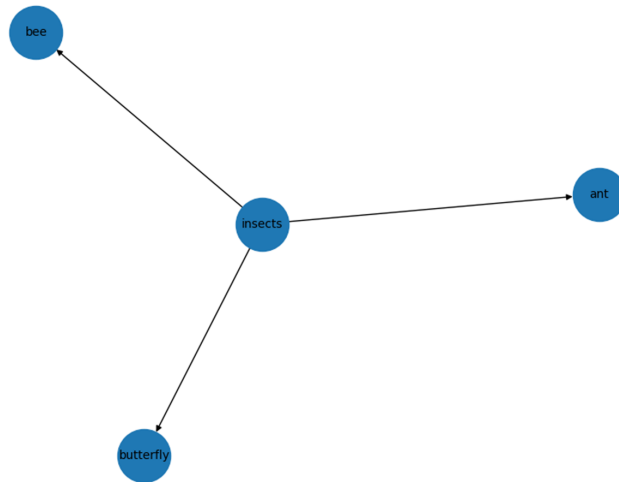


Figura 13: Mapa Conceptual de Insectos: Relación entre insectos como "bee", "butterfly", y "ant".



Figura 14: Mapa Conceptual Global: Integración de conceptos de frutas e insectos bajo un mapa global.

9. Conclusiones

El sistema ha demostrado ser efectivo para identificar relaciones de **hiperonimia** e **hiponimia** en textos. A través de diferentes enfoques, como expresiones regulares, análisis semántico y análisis de dependencias, se logró generar mapas conceptuales a partir de un texto de entrada. Sin embargo, existen áreas de mejora en cuanto a la cohesión y optimización del proceso, particularmente en la precisión del análisis semántico y la eficiencia en textos más largos.